

# Homework 1 — Tokenizer pruning for caveman English

Ilia Mikhalechuk · Modern Methods and Algorithms of Generative AI · Skoltech, Fall 2026

Code: [github.com/miilv/prune-to-caveman](https://github.com/miilv/prune-to-caveman) · Pruned model: [huggingface.co/miilv/qwen2.5-0.5b-caveman](https://huggingface.co/miilv/qwen2.5-0.5b-caveman)

## 1. Language, corpus and setup

**Language.** *Caveman English*, a closed-vocabulary register of English: lowercase base forms from a list of 961 words (Basic English 850 + 111 caveman words such as *mammoth*, *spear*, *tribe*, *kill*, *hunt*, minus *a* and *the*), no inflections, about 140 frequent proper names (plus the names in the article’s own title) kept Capitalized, digits and . , ? !. Example: “*Elephantomyia be dead kind of long leg fly, from long ago. people know it only from clear tree stone.*” (amber). A synthetic register rather than a natural language, chosen as the extreme case of the homework’s setting.

**Corpus.** English Wikipedia ([wikimedia/wikipedia 20231101.en](https://wikimedia/wikipedia/20231101.en), two shards drawn with seed 0, CC BY-SA 4.0); articles on history, nature, places, science, myth and crafts (topic labelled by an LLM, modern topics dropped), split by article into train and held-out before generation, then rewritten paragraph by paragraph by **claude-haiku-5-5**. Every output was checked word by word; 4.0% of words broke the rules in the first pass, 1.04% after one repair turn; the rest was fixed by a corpus-wide replacement dictionary and 0.42% of sentences were dropped. **Train:** 2,003 articles, 9.17 MB, 1,841,169 words. **Held-out:** 182 other articles, 793 KB (all of it is evaluated). The English originals of the held-out articles (1163 KB) are used only as an *out-of-language probe* (first 4,000 chars of 60 documents).

**Setup.** Qwen/Qwen2.5-0.5B (byte-level BPE, 151,643 + 22 added tokens, tied embeddings), seed 0, one NVIDIA GeForce RTX 4090, bfloat16 weights with float32 cross-entropy, torch 2.14.1+cu130, transformers 5.19.0, tokenizers 0.23.2. The whole notebook runs in 1.6 min on this GPU (on a T4 it switches to fp16, which gives the same bits per byte to 4 decimals). Bits per byte follows §5 of the assignment exactly (<|endoftext|> prefix, non-overlapping windows of 512 tokens); my batched implementation is checked against the starter’s function in the notebook.

## 2. Token usage (Part 1)

| tokenizer           | vocab   | bytes/token | fertility | distinct ids | share  | ids for 99% |
|---------------------|---------|-------------|-----------|--------------|--------|-------------|
| Qwen2.5 (151k)      | 151,643 | 3.87        | 1.289     | 4,973        | 3.28%  | 1,918       |
| GPT-2 r50k (50k)    | 50,257  | 4.05        | 1.230     | 5,995        | 11.93% | 2,474       |
| GPT-4 cl100k (100k) | 100,277 | 4.04        | 1.233     | 5,961        | 5.94%  | 2,329       |
| GPT-4o o200k (200k) | 200,019 | 4.05        | 1.230     | 6,287        | 3.14%  | 2,419       |

Table 1: Train corpus (9.2 MB). Fertility = tokens per whitespace word.

Qwen2.5 needs only 4,973 of its 151,643 ids (3.28%) for the whole train corpus, and 1,918 ids cover 99% of all token occurrences (Figure 1). On the held-out side the same tokenizer uses 2,018 ids for the caveman text and 21,867 for the English originals of the **same** articles. All four tokenizers are practically equivalent here (fertility 1.23–1.29): every list word is one token with its leading space, the rest are punctuation, digits and pieces of rare names. Qwen is slightly worse only because its pre-tokenizer splits numbers into single digits and caveman text is full of years (“year 1453”): with the numbers removed Qwen and cl100k both give 1.172. A bigger vocabulary buys nothing for this language.

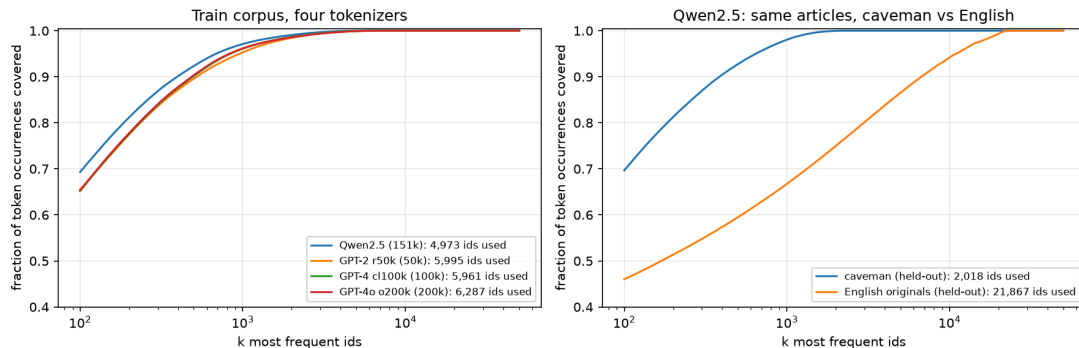


Figure 1: Coverage of token occurrences by the  $k$  most frequent ids. Left: train corpus, four tokenizers. Right: Qwen2.5 on the held-out articles, caveman vs their English originals.

## 3. Pruning the tokenizer (Part 2)

With  $m = 1$  the vocabulary shrinks from 151,643 to **6,244** tokens and the merges from 151,387 to **5,988** (22 added tokens kept, renumbered after the vocabulary; 1,083 tokens are added only by the closure). Checks: (1) `decode(encode(x)) == x` holds for all held-out documents and also for the English originals (the 256 byte-level characters are always kept); (2) all 2,003 train articles are tokenized identically; (3) on held-out text 40.7% of the articles (78.1% of the paragraphs) are identical, length ratio 1.0106 — the differences are names that never occurred in train; the English originals: length ratio 1.523.

**Why (2) must hold.** Pre-tokenization does not depend on the vocabulary. Inside a pre-token BPE always applies the applicable merge of lowest rank. Every token produced while tokenizing the train corpus is either final (count  $\geq 1$ , so kept) or an intermediate that is later merged into a final token; since Qwen’s merges have unique results, these intermediates are exactly the nodes of the final token’s merge tree, which the closure keeps. So every merge that ever fired survives in the same order, and the removed merges never fired: at every step the pruned tokenizer makes the same choice.

**Closure example.** `␣this` (merge #163) is built as `␣th + is`, and `␣th = ␣t + h`. Neither `␣th` nor `␣t` ever occurs as a final token, so a frequency-only vocabulary drops them. Then **tokenizers** refuses to load the file (“*Token 6t out of vocabulary*”); if we also drop the merges with missing inputs, `␣this` can no longer be built and becomes `␣ + this`. In total 1,794 kept tokens become unreachable dead rows, no train article is tokenized identically any more and sequences get  $1.92\times$  longer.

## 4. Pruning the model (Part 3)

|                    | params  | bf16 checkpoint | bits/byte | tokens/byte | bits/byte EN probe | tokens/byte EN probe |
|--------------------|---------|-----------------|-----------|-------------|--------------------|----------------------|
| original           | 494.0 M | 1000 MB         | 1.2846    | 0.2607      | 0.970              | 0.2383               |
| pruned $m = 1$     | 363.5 M | 727 MB          | 1.2279    | 0.2628      | 2.102              | 0.3571               |
| reloaded from disk | 363.5 M | 727 MB          | 1.2279    | 0.2628      |                    |                      |

Table 2: Held-out caveman (206,666 tokens, 793 KB) and the English probe. Embedding rows: 151,936  $\rightarrow$  6,272 (padded to 64).

Embeddings are sliced by index (no `resize_token_embeddings`), the tied head shares the new matrix, `vocab_size` and the special-token ids in `config / generation_config` are remapped (otherwise `generate` stops on a wrong id). Parameters drop by 26.4% and the checkpoint by 27.2%. Greedy generation (float32) of the pruned model is identical to the original’s as long as the original’s choice is still in the vocabulary (1 of 3 prompts; the other two diverge at `␣pact` and `aming`, which were pruned); the reloaded checkpoint generates the same text and reproduces bits per byte exactly.

**Why bits per byte improves (1.2846  $\rightarrow$  1.2279) and why the model is not better.** The pruned head is a softmax over 6,266 logits instead of 151k; all kept logits are unchanged, so for every target token that is still in the vocabulary  $p_{\text{pruned}}(t) = \frac{p(t)}{\sum_{t' \text{ kept}} p(t')} \geq p(t)$ . The mass the model put on 145,399 removed tokens (other scripts, code, rare English) is redistributed over the kept ones, and bits per byte, which scores the exact token sequence, drops by the log of that factor. Nothing in the network changed — the same number comes from masking the logits — we only injected a prior from train-corpus statistics about which tokens can occur. The price shows on text outside the domain: on ordinary English the pruned model needs  $1.50\times$  more tokens and bits per byte rises from 0.970 to 2.102.

## 5. How far can we go? (Part 4)

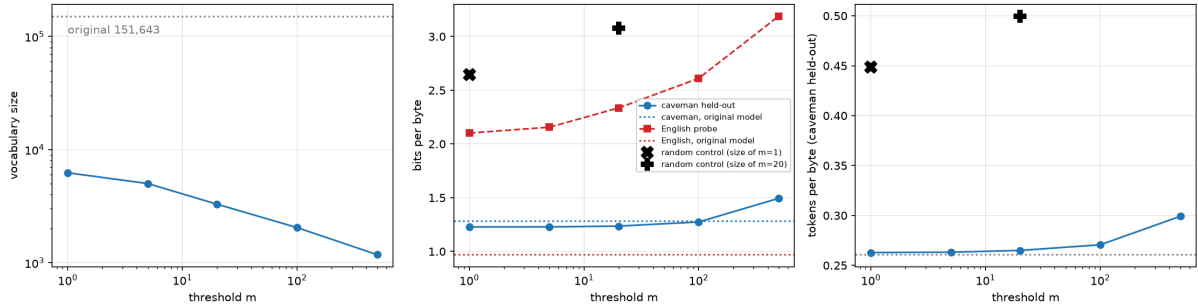


Figure 2: Threshold sweep. Black markers: random control with the same vocabulary size (closure respected).

| run                    | vocab   | params, M | held identical | length ratio | tokens/byte | bits/byte | EN length ratio | EN bits/byte |
|------------------------|---------|-----------|----------------|--------------|-------------|-----------|-----------------|--------------|
| original               | 151,643 | 494.0     | 100.0%         | 1.0000       | 0.2607      | 1.2846    | 1.000           | 0.970        |
| $m=1$                  | 6,244   | 363.5     | 40.7%          | 1.0106       | 0.2628      | 1.2279    | 1.507           | 2.102        |
| $m=5$                  | 5,012   | 362.4     | 29.1%          | 1.0134       | 0.2632      | 1.2287    | 1.539           | 2.156        |
| $m=20$                 | 3,296   | 360.9     | 6.6%           | 1.0207       | 0.2650      | 1.2358    | 1.622           | 2.335        |
| $m=100$                | 2,045   | 359.8     | 1.1%           | 1.0423       | 0.2706      | 1.2742    | 1.737           | 2.611        |
| $m=500$                | 1,178   | 359.0     | 0.0%           | 1.1488       | 0.2992      | 1.4949    | 1.967           | 3.190        |
| random, size of $m=1$  | 6,244   | 363.5     | 0.0%           | 1.7211       | 0.4486      | 2.6450    | 1.911           | 2.844        |
| random, size of $m=20$ | 3,296   | 360.9     | 0.0%           | 1.9164       | 0.4998      | 3.0777    | 2.141           | 3.381        |

Table 3: Threshold sweep and random control on the held-out caveman text and the English probe (EN).

**Random control.** Keeping the same number of tokens at random (with closure) instead of the frequent ones makes the caveman text  $1.72\times$  longer and bits per byte jumps to 2.645 (vs 1.2279). The gain is therefore not “fewer tokens are fine”: it comes from the extremely skewed usage distribution, which the frequency threshold follows.

**Knee and what to ship.** Bits per byte stays below the original up to  $m = 100$  (vocabulary  $\approx 2k$ : roughly the 961 words in their 1–2 spellings plus punctuation and digits) and breaks at  $m = 500$ , where list words start to split. In **parameters**, however, the knee is at  $m = 1$  already: the embedding share falls from 27.6% to 1.6%, and every further step saves at most 1.3% of the model while the held-out agreement drops from 40.7% to 6.6% ( $m = 20$ ). I would ship  **$m = 1$** :  $-26.4\%$  parameters, the best bits per byte, exactly the original tokenization on everything seen in training, and the most robust handling of unseen names.

## 6. Bonuses

**B3 — tokenizer trained from scratch.** A byte-level BPE trained on the caveman corpus with Qwen’s pre-tokenizer and the same vocabulary size gives almost the same tokens/byte, but plugged into Qwen with random embeddings the model is useless: 4.719 bits/byte — worse than a uniform distribution — and generation degenerates into repeating one token. Initialising each new token as the mean of the Qwen embeddings of its pieces (FVT) restores 1.232 bits/byte without any training. **B1 — vocabulary extension.** New merges trained with **tokenizers** on the caveman corpus, appended after the pruned vocabulary (ranked by how often their inputs are adjacent), mean-initialised: fertility 1.2889  $\rightarrow$  1.2841 with 500 new tokens — little room is left, since every list word is already one token and only rare names split. An embedding-only fine-tune (100 steps) lowers bits per byte to  $\approx 1.198$  for the pruned and the extended model alike. **B4 — serving.** Peak memory 1002  $\rightarrow$  747 MiB at batch 1; decode throughput changes by only  $+2\%$  (batch 1): GPU decoding of a 0.5B model is dominated by the 24 layers and kernel launches, not by the head.

## 7. Limitations

The corpus is LLM-generated: its style is Haiku’s, it is shorter than the source (caveman/English  $\approx 0.68$ ) and some paragraphs are condensed; held-out text comes from the same generator, so it is in-distribution by construction. Caveman English is a register of English, not a natural language, and its closed vocabulary makes pruning much cleaner than for a real language. Re-generating the corpus would not reproduce it byte for byte (hence the frozen files). One model, one seed, one corpus. The original model does not “speak” caveman (it continues prompts in normal English); pruning restricts the vocabulary but not the grammar.

**AI assistance.** The notebook, the corpus pipeline and this report were written with an AI coding assistant (Claude Code with Claude Opus 5.5) under my direction; the corpus itself was generated by **claude-haiku-5-5** ( $\approx 47$  M output tokens).

## 8. Appendix

| new tokens | fertility (train) | tokens/byte (held) | bits/byte (held) | after embedding fine-tune |
|------------|-------------------|--------------------|------------------|---------------------------|
| 0          | 1.2889            | 0.2628             | 1.2279           | 1.1979                    |
| 100        | 1.2865            | 0.2628             | 1.2287           | 1.1970                    |
| 500        | 1.2841            | 0.2627             | 1.2299           | 1.1831                    |

Table 4: B1 — vocabulary extension on top of the  $m = 1$  tokenizer. First new tokens: `␣Gowa`, `atchez`, `ruze`, `inteler`, `␣Cus`, `␣Neculu`, `␣Wint`, `␣Assam`.

| model      | batch | tokens/s | ms/step | peak MiB |
|------------|-------|----------|---------|----------|
| original   | 1     | 166      | 6.01    | 1002     |
| original   | 8     | 1059     | 7.55    | 1037     |
| pruned m=1 | 1     | 170      | 5.87    | 747      |
| pruned m=1 | 8     | 1104     | 7.25    | 762      |

Table 5: B4 — greedy decoding of 128 new tokens, NVIDIA GeForce RTX 4090, bfloat16.

**Corpus generation.** 3,482 chunks of  $\leq 6,000$  characters; 98.3% accepted after at most one repair turn (6 chunks never produced an answer and are missing);  $\approx 90\%$  of the 47 M output tokens were the model’s thinking; prompt caching served 77% of the 24 M input tokens. With thinking disabled 15–30% of the words broke the rules and repair turns did not converge, so thinking was kept on. The full pipeline (`corpus_pipeline/`) and the style guide given to the model are in the repository.

**Reproduce.** Open `hw1_mikhailchuk.ipynb` in Colab (GPU) or any machine with a GPU and run all cells; it downloads the frozen corpus from the repository and Qwen from the Hub. Runtime per part (s, NVIDIA GeForce RTX 4090): corpus 0, part1 2, part2 3, part3 12, part4 27, ship 0, B3 5, B1 40, B4 8.